ASSIGNMENT – 10

NAME : SATHWIK REDDY

HALL TICKET NO : 2303A51989

BATCH : 15

Question :

Exploring Layer 2 – Testnet

Objective To understand and explore Layer 2 blockchain networks by deploying and interacting with a simple smart contract on a Layer 2 Testnet (e.g., Polygon Mumbai / Arbitrum Sepolia), and observe reduced gas fees and faster transactions compared to Layer

Requirements :

• Basic understanding of blockchain and smart contracts

• MetaMask wallet installed (browser extension)

• Testnet ETH/MATIC from faucet

• VS Code (with Solidity extension installed)

• Node.js and npm installed

• Hardhat development framework Tools & Technologies Used

• Blockchain Type: Layer 2 Testnet

• Network: Polygon Mumbai Testnet (or Arbitrum Sepolia)

Language: Solidity

Framework: Hardhat

Wallet: MetaMask

Execution Key is:- 0x742d35Cc6634C0532925a3b844Bc454e4438f44e

CODE :

```
import tkinter as tk from tkinter
import messagebox import
random import time class DeFi:
        def __init__(self):
self.balances = {} def
gas_fee(self):
```

```
        return round(random.uniform(0.0001, 0.001), 6)
def delay(self):     time.sleep(0.3) def deposit(self,
user, amount):        if amount <= 0:
            return "Invalid amount"
        self.delay()        gas = self.gas_fee()        if user not in self.balances:
self.balances[user] = 0        self.balances[user] += amount        return f"Deposit
Success\nUser: {user}\nAmount: {amount} ETH\nGas: {gas}"     def withdraw(self,
user, amount):        if user not in self.balances:            return "User not found"
if self.balances[user] < amount:            return "Insufficient balance"        self.delay()
gas = self.gas_fee()        self.balances[user] -= amount        return f"Withdraw
Success\nUser: {user}\nAmount: {amount} ETH\nGas: {gas}"
    def balance(self, user):
        return self.balances.get(user, 0)
class App:
    def __init__(self, root):
        self.defi = DeFi()        self.root = root
self.root.title("DeFi Dashboard")
self.root.geometry("600x400")        left_frame =
tk.Frame(root, bg="#2c3e50", width=200)
left_frame.pack(side="left", fill="y")     tk.Label(left_frame,
text="DeFi Panel", fg="white",            bg="#2c3e50",
font=("Arial", 14, "bold")).pack(pady=20)
tk.Label(left_frame, text="User", fg="white",
bg="#2c3e50").pack()     self.user_entry =
tk.Entry(left_frame)        self.user_entry.pack(pady=5)
tk.Label(left_frame, text="Amount", fg="white",
bg="#2c3e50").pack()        self.amount_entry =
tk.Entry(left_frame)        self.amount_entry.pack(pady=5)
tk.Button(left_frame, text="Deposit", width=15,
command=self.deposit).pack(pady=10)
tk.Button(left_frame, text="Withdraw", width=15,
command=self.withdraw).pack(pady=5)
tk.Button(left_frame, text="Check Balance", width=15,
command=self.check_balance).pack(pady=5)
```

```
right_frame = tk.Frame(root, bg="#ecf0f1")
right_frame.pack(side="right", expand=True, fill="both")
tk.Label(right_frame, text="Transaction Output",
bg="#ecf0f1", font=("Arial", 14, "bold")).pack(pady=10)
self.output = tk.Text(right_frame, height=15, width=50)
self.output.pack(padx=10, pady=10)     def deposit(self):
        user = self.user_entry.get()
        try:
            amount =
float(self.amount_entry.get())            result =
self.defi.deposit(user, amount)
self.show(result)        except:
            messagebox.showerror("Error", "Invalid input")
def withdraw(self):
        user = self.user_entry.get()
        try:
            amount = float(self.amount_entry.get())
    result = self.defi.withdraw(user, amount)
    self.show(result)     except:
            messagebox.showerror("Error", "Invalid input")
def check_balance(self):
        user = self.user_entry.get()        balance =
self.defi.balance(user)        self.show(f"{user}
Balance: {balance} ETH")     def show(self, text):
        self.output.delete(1.0, tk.END)
self.output.insert(tk.END, text)
if __name__ == "__main__":
    root = tk.Tk()
app = App(root)
root.mainloop()
```

IMPLEMENTATION :

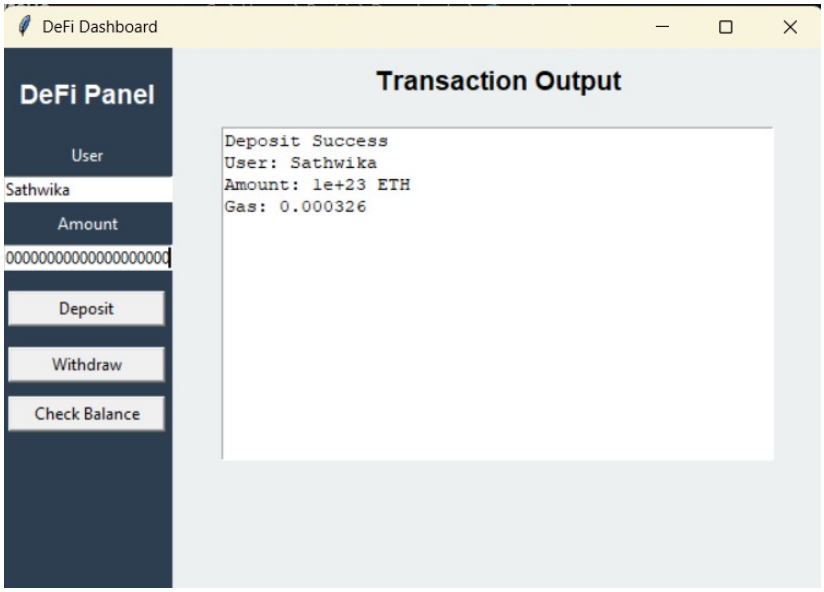
C: > Users > Rushi > Downloads > gui.py > DeFi

```python
import tkinter as tk
from tkinter import messagebox
import random
import time
class DeFi:
    def __init__(self):
        self.balances = {}
    def gas_fee(self):
        return round(random.uniform(0.0001, 0.001), 6)
    def delay(self):
        time.sleep(0.3)
    def deposit(self, user, amount):
        if amount <= 0:
            return "Invalid amount"
        self.delay()
        gas = self.gas_fee()
        if user not in self.balances:
            self.balances[user] = 0
        self.balances[user] += amount
        return f"Deposit Success\nUser: {user}\nAmount: {amount} ETH\nGas: {gas}"
    def withdraw(self, user, amount):
        if user not in self.balances:
            return "User not found"
        if self.balances[user] < amount:
            return "Insufficient balance"
        self.delay()
        gas = self.gas_fee()
        self.balances[user] -= amount
        return f"Withdraw Success\nUser: {user}\nAmount: {amount} ETH\nGas: {gas}"
    def balance(self, user):
        return self.balances.get(user, 0)
class App:
    def __init__(self, root):
```

C: > Users > Rushi > Downloads > gui.py > DeFi

```python
class App:
    def __init__(self, root):
        tk.Label(right_frame, text="Transaction Output",
                 bg="#ecf0f1", font=("Arial", 14, "bold")).pack(pady=10)
        self.output = tk.Text(right_frame, height=15, width=50)
        self.output.pack(padx=10, pady=10)
    def deposit(self):
        user = self.user_entry.get()
        try:
            amount = float(self.amount_entry.get())
            result = self.defi.deposit(user, amount)
            self.show(result)
        except:
            messagebox.showerror("Error", "Invalid input")
    def withdraw(self):
        user = self.user_entry.get()
        try:
            amount = float(self.amount_entry.get())
            result = self.defi.withdraw(user, amount)
            self.show(result)
        except:
            messagebox.showerror("Error", "Invalid input")
    def check_balance(self):
        user = self.user_entry.get()
        balance = self.defi.balance(user)
        self.show(f"{user} Balance: {balance} ETH")
    def show(self, text):
        self.output.delete(1.0, tk.END)
        self.output.insert(tk.END, text)
if __name__ == "__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()
```

OUTPUT :


DeFi Dashboard
DeFi Panel
User
Sathwika
Amount
000000000000000000000
Deposit
Withdraw
Check Balance
Transaction Output
Deposit Success
User: Sathwika
Amount: 1e+23 ETH
Gas: 0.000326